C++ Midway Project

Inventory Management System

John Saldivar

Project Type	Command-line C++ application
Storage	inventory.txt using file input/output
Search	Binary search by unique SKU
Memory	Dynamically allocated Product array
Security	Admin and Regular User roles
Validation	Types, ranges, required fields, dates

This report documents the completed Inventory Management System and includes screenshots from compiled program test runs demonstrating the required course concepts and application features.

1. Program Overview

The Inventory Management System is a multi-file C++ application that stores and manages product records. Each product has a unique SKU, product name, quantity, price, and date added. The program loads records from a text file when it starts, keeps the records sorted by SKU, allows authorized users to add or delete records, searches records using binary search, validates user input, and saves changes back to the text file.

The project also demonstrates dynamic memory management. The InventorySystem class allocates a Product array with **new[]** and releases it with **delete[]**. The Product pointer is used for array access and is displayed by the program as part of the pointer demonstration.

How the Program Works

```
Program Start
    |
    v
Load inventory.txt
    |
    v
Create Admin / Regular User objects
    |
    v
Choose Role
    |
    v
Validated Menu
    |
    +--> View sorted inventory
    +--> Binary search by SKU
    +--> Add validated Product (Admin)
    +--> Delete Product (Admin)
    +--> Save inventory.txt
    +--> Show data type sizes
    +--> Show pointer / dynamic memory demo
    +--> Show required C++ concepts
    |
    v
Save file and exit
```

Project Files

```
Midterm/
|-- main.cpp
|-- Product.h / Product.cpp
|-- User.h / User.cpp
|-- InventorySystem.h / InventorySystem.cpp
|-- InputValidator.h / InputValidator.cpp
|-- inventory.txt
|-- README.md
`-- output.pdf
```

2. Required C++ Concepts Used

Required concept	How it is demonstrated
Data types and data sizes	int, double, bool, char, std::string, Product* and sizeof() output
File types	inventory.txt stores persistent inventory records
Pointers	Product* products and User* currentUser
Dynamic memory	new Product[capacity] and delete[] products
Arrays	Dynamically allocated Product array; fixed validation array
Binary search	InventorySystem::binarySearchBySku()
Strings	Names, usernames, dates, parsing, file records
File I/O	std::ifstream loads records; std::ofstream saves records
At least two classes	Product, InventorySystem, User, Admin, RegularUser, InputValidator

Additional Concepts

The project additionally demonstrates inheritance, polymorphic role interfaces, public/private/protected access, encapsulation, constructors/destructors, input validation, sorted insertion, multi-file organization, and exception-safe handling of malformed text-file records.

File Format

Each line of **inventory.txt** uses the following format:

```
SKU|Product Name|Quantity|Price|Date Added
1001|Brake Pads|5|39.99|10/01/2026
```

3. Program Startup and Inventory Display

Startup, Admin role, menu, and inventory display

```
==================================================
          C++ MIDWAY PROJECT
          INVENTORY MANAGEMENT SYSTEM
==================================================
John Saldivar

Inventory data loaded from inventory.txt.

Choose a role:
1. Admin
2. Regular User
Selection:
Logged in as: admin_user (Admin)

======================================
             INVENTORY MENU
======================================
1. View Records
2. Search Record by SKU
3. Add Record
4. Delete Record
5. Save Inventory to File
6. Show Data Type Sizes
7. Show Pointer / Dynamic Memory Demo
8. Show Required Concepts
9. Exit
Select an option:
=====================================================================
                        INVENTORY RECORDS
=====================================================================
SKU       Product                 Quantity    Price       Date Added
---------------------------------------------------------------------
1001      Brake Pads              5           $39.99      10/01/2026
1002      Oil Filter              10          $8.99       10/02/2026
1005      Air Filter              7           $14.49      10/03/2026
1010      Spark Plugs             12          $6.25       10/04/2026
```

The program loads inventory.txt, allows role selection, shows the menu, and displays the sorted Product records.

4. Binary Search

```
Binary search: found and not found
```

```
========================================
            INVENTORY MENU
========================================
1. View Records
2. Search Record by SKU
3. Add Record
4. Delete Record
5. Save Inventory to File
6. Show Data Type Sizes
7. Show Pointer / Dynamic Memory Demo
8. Show Required Concepts
9. Exit
Select an option: Enter SKU to search for (1-999999):
Product found using binary search:
SKU       Product                Quantity    Price       Date Added
------------------------------------------------------------------------
1005      Air Filter             7           $14.49      10/03/2026

========================================
            INVENTORY MENU
========================================
1. View Records
2. Search Record by SKU
3. Add Record
4. Delete Record
5. Save Inventory to File
6. Show Data Type Sizes
7. Show Pointer / Dynamic Memory Demo
8. Show Required Concepts
9. Exit
Select an option: Enter SKU to search for (1-999999):
No product found with SKU 9999.
```

Binary search successfully finds SKU 1005 and correctly reports that SKU 9999 does not exist.

5. Input Validation and Sorted Insertion

```
Input validation, retry messages, and sorted record insertion
```

```
========================================
            INVENTORY MENU
========================================
1. View Records
2. Search Record by SKU
3. Add Record
4. Delete Record
5. Save Inventory to File
6. Show Data Type Sizes
7. Show Pointer / Dynamic Memory Demo
8. Show Required Concepts
9. Exit
Select an option: Enter SKU (1-999999): Invalid range. Enter a value from 1 to 999999.
Enter SKU (1-999999): Invalid input. Please enter a whole number.
Enter SKU (1-999999): Invalid SKU. That SKU already exists. Please try a different SKU.
Enter SKU (1-999999): Enter product name: Invalid input. This field is required and cannot be
    empty.
Enter product name: Enter quantity (0-1000000): Invalid range. Enter a value from 0 to 1000000.
Enter quantity (0-1000000): Enter price (0.01-1000000.00): $Invalid range. Enter a value from
    0.01 to 1e+06.
Enter price (0.01-1000000.00): $Enter date added (MM/DD/YYYY): Invalid date. Use MM/DD/YYYY and
    enter a real date between 2000 and 2100.
Enter date added (MM/DD/YYYY):
Record added successfully in sorted SKU order.
Inventory automatically saved to inventory.txt.

========================================
            INVENTORY MENU
========================================
1. View Records
2. Search Record by SKU
3. Add Record
4. Delete Record
5. Save Inventory to File
6. Show Data Type Sizes
7. Show Pointer / Dynamic Memory Demo
8. Show Required Concepts
9. Exit
Select an option:
========================================================================
                       INVENTORY RECORDS
========================================================================
SKU       Product                Quantity    Price       Date Added
------------------------------------------------------------------------
1001      Brake Pads             5           $39.99      10/01/2026
1002      Oil Filter             10          $8.99       10/02/2026
1003      Engine Cleaner         4           $12.50      11/01/2026
1005      Air Filter             7           $14.49      10/03/2026
1010      Spark Plugs            12          $6.25       10/04/2026
```

The application rejects invalid role, SKU, empty-name, negative quantity, invalid price, and invalid-date input. After valid data is entered, SKU 1003 is inserted in sorted order.

6. File I/O Persistence

File I/O persistence after restarting the program

```
==================================================
          C++ MIDWAY PROJECT
          INVENTORY MANAGEMENT SYSTEM
==================================================
John Saldivar

Inventory data loaded from inventory.txt.

Choose a role:
1. Admin
2. Regular User
Selection:
Logged in as: admin_user (Admin)

========================================
             INVENTORY MENU
========================================
1. View Records
2. Search Record by SKU
3. Add Record
4. Delete Record
5. Save Inventory to File
6. Show Data Type Sizes
7. Show Pointer / Dynamic Memory Demo
8. Show Required Concepts
9. Exit
Select an option:
======================================================================
                        INVENTORY RECORDS
======================================================================
SKU       Product                 Quantity    Price       Date Added
----------------------------------------------------------------------
1001      Brake Pads              5           $39.99      10/01/2026
1002      Oil Filter              10          $8.99       10/02/2026
1003      Engine Cleaner          4           $12.50      11/01/2026
1005      Air Filter              7           $14.49      10/03/2026
1010      Spark Plugs             12          $6.25       10/04/2026
```

After a Product is saved, restarting the program reloads SKU 1003 from inventory.txt, demonstrating persistent file input/output.

7. Data Types and Data Sizes

```
Data types and data sizes using sizeof()

=======================================
            DATA TYPE SIZES
=======================================
sizeof(char)      = 1 byte(s)
sizeof(bool)      = 1 byte(s)
sizeof(int)       = 4 byte(s)
sizeof(double)    = 8 byte(s)
sizeof(Product*)  = 8 byte(s)
```

The program uses sizeof() to display the platform data sizes for several C++ types and a Product pointer.

8. Pointers and Dynamic Memory

```
Pointer and dynamically allocated Product array

========================================
          POINTER / DYNAMIC ARRAY
========================================
The inventory is stored in a dynamically
allocated Product array.

Product array pointer address: 0x557886c222c8
Current records: 4
Dynamic array capacity: 50
First SKU through pointer access: 1001
```

The InventorySystem stores Products in a dynamically allocated Product array. This screen displays the Product pointer address, record count, capacity, and pointer-based access to the first SKU.

9. Required Concepts Screen

```
Required C++ concepts demonstration screen

========================================
       REQUIRED C++ CONCEPTS USED
========================================
Data types and data sizes      : Yes
Text file (.txt)               : Yes
Pointers                       : Yes
Dynamic memory                 : Yes
Arrays                         : Yes
Binary search                  : Yes
Strings                        : Yes
File input/output              : Yes
Two or more classes            : Yes
Input validation               : Yes
Role-based permissions         : Yes
```

A built-in project screen summarizes the major required C++ concepts demonstrated by the application.

10. Regular User Permissions

```
Regular User permissions and binary search

========================================
            INVENTORY MENU
========================================
1. View Records
2. Search Record by SKU
3. Add Record
4. Delete Record
5. Save Inventory to File
6. Show Data Type Sizes
7. Show Pointer / Dynamic Memory Demo
8. Show Required Concepts
9. Exit
Select an option:
Access denied: Regular Users cannot add records.

========================================
            INVENTORY MENU
========================================
1. View Records
2. Search Record by SKU
3. Add Record
4. Delete Record
5. Save Inventory to File
6. Show Data Type Sizes
7. Show Pointer / Dynamic Memory Demo
8. Show Required Concepts
9. Exit
Select an option:
Access denied: Regular Users cannot delete records.

========================================
            INVENTORY MENU
========================================
1. View Records
2. Search Record by SKU
3. Add Record
4. Delete Record
5. Save Inventory to File
6. Show Data Type Sizes
7. Show Pointer / Dynamic Memory Demo
8. Show Required Concepts
9. Exit
Select an option: Enter SKU to search for (1-999999):
Product found using binary search:
SKU       Product                Quantity    Price       Date Added
---------------------------------------------------------------------
1005      Air Filter             7           $14.49      10/03/2026
```

A Regular User can view and search records but receives access-denied messages when attempting to add or delete records.

11. Add, Delete, and Save Operations

```
Admin add/delete operations and file save

Select an option: Enter SKU (1-999999): Enter product name: Enter quantity (0-1000000): Enter
    price (0.01-1000000.00): $Enter date added (MM/DD/YYYY):
Record added successfully in sorted SKU order.
Inventory automatically saved to inventory.txt.

========================================
             INVENTORY MENU
========================================
1. View Records
2. Search Record by SKU
3. Add Record
4. Delete Record
5. Save Inventory to File
6. Show Data Type Sizes
7. Show Pointer / Dynamic Memory Demo
8. Show Required Concepts
9. Exit
Select an option: Enter SKU to delete (1-999999):
Record deleted successfully.
Inventory automatically saved to inventory.txt.

========================================
             INVENTORY MENU
========================================
1. View Records
2. Search Record by SKU
3. Add Record
4. Delete Record
5. Save Inventory to File
6. Show Data Type Sizes
7. Show Pointer / Dynamic Memory Demo
8. Show Required Concepts
9. Exit
Select an option:
Inventory saved successfully to inventory.txt.
```

The Admin can add a Product, delete it by SKU, and explicitly save the inventory to inventory.txt.

12. Build, Testing, and Conclusion

Build Command

```
g++ -std=c++17 -Wall -Wextra -pedantic *.cpp -o inventory_manager
```

Testing Completed

- Compilation with C++17 and warning flags
- File loading and saving
- Sorted Product display
- Binary search success and not-found cases
- Admin add and delete operations
- Regular User permission restrictions
- Invalid type and range input
- Required Product name validation
- Date format and real calendar-date validation
- Dynamic array allocation and pointer access
- Persistence after program restart
- Normal program exit

Conclusion

The completed Inventory Management System combines the major C++ concepts covered during the first half of the course into one working application. It uses classes to organize the program, a dynamically allocated Product array for storage, pointers for memory access, binary search for efficient lookup, text-file I/O for persistent data, strings and multiple numeric data types, and validation to protect the stored information. The project also preserves sorted records and role-based permissions while remaining organized across reusable header and implementation files.

Grading area	Project evidence
Required C++ features	All listed concepts are explicitly implemented and demonstrated
Working functionality	Compiled and tested through Admin and Regular User workflows
Clear output.pdf	Overview, concept list, architecture, and program-run screenshots
Program structure	Separate .h/.cpp files for Products, users, inventory, and validation
Readability/comments	Descriptive names, organized functions/classes, targeted code comments